Accretion

SECURITY ASSESSMENT

Provided by Accretion Labs Pte Ltd. for Light Protocol
March 05, 2026
A25LIG5

Light Protocol

AUDITORS

Role	Name
Lead Auditor	Robert Reith (robert@accretion.xyz)
Auditor	Mahdi Rostami (mahdi@accretion.xyz)

CLIENT

Light Protocol (https://lightprotocol.com) engaged Accretion to conduct a comprehensive security audit of their Compressed Token Program V2 implementation their compression layer on Solana.

ENGAGEMENT TIMELINE

- 01 Sep — **Project Kickoff**
  Initial planning and scope definition
- 01 Sep — **Assessment Begins**
  Security review and testing phase
- 01 Dec — **Review Fixes**
  Security recommendations are given and implemented
- 05 Mar — **Project Completion**
  Report delivery and on-chain confirmation

AUDITED CODE

#	Program ID	Repository
1	compr6CUsB5m2jS4Y3831ztGSTnDpnKJTKS95d64XVq	https://github.com/Lightprotocol/light-protocol
2	SySTEM1eSU2p4BGQfQpimFEWWSC1XDFeun3Nqzz3rT7	https://github.com/Lightprotocol/light-protocol
3	Lighton6oQpVkeewmo2mcPTQQp7kYHr4fWpAgJyEmDX	https://github.com/Lightprotocol/light-protocol
4	cTokenmWW8bLPjZEBAUgYy3zKxQZW6VKi7bqNFEVv3m	https://github.com/Lightprotocol/light-protocol

ASSESSMENT

The security assessment of Light Protocol's compressed-token-program-v2 revealed 21 issues across critical (1), high (3), medium (11), low (2), and informational (4) severity levels. All identified vulnerabilities have been successfully fixed by the development team, demonstrating strong commitment to security.

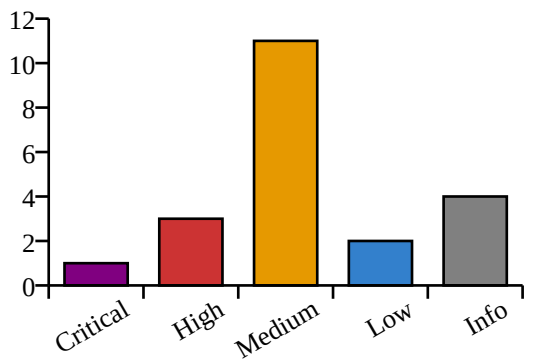
CODE ASSESSMENT

The codebase is a Pinocchio rewrite of an existing Anchor program with complex token compression functionality. The code shows sophisticated architecture but initially lacked critical validation checks and proper access controls in key operations.

KEY FINDINGS

Critical findings include an Anchor IDL buffer feature allowing arbitrary token account creation. High-severity issues involved output address uniqueness bypass, batched address tree rollover fee redirection enabling fee-payer draining, and queue indexing failures causing all inserts to target queue 0.

SEVERITY DISTRIBUTION



© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ENGAGEMENT SCOPE

The scope of this security assessment was a full review of the following items:

Item 1: Compression Program
Link: https://github.com/Lightprotocol/light-protocol
Commit: 13c64a1f3c6205333aa8f40554f35b2aed5d78ca
Program ID: compr6CUsB5m2jS4Y3831ztGSTnDpnKJTKS95d64XVq
Audit Result:
• **Audited Commit:** 1ee21c0976876def2e6f574116baa1e82c52d7f5

Item 2: System Program
Link: https://github.com/Lightprotocol/light-protocol
Commit: 13c64a1f3c6205333aa8f40554f35b2aed5d78ca
Program ID: SySTEM1eSU2p4BGQfQpimFEWWSC1XDFeun3Nqzz3rT7
Audit Result:
• **Audited Commit:** 1ee21c0976876def2e6f574116baa1e82c52d7f5

Item 3: Registry Program
Link: https://github.com/Lightprotocol/light-protocol
Commit: 13c64a1f3c6205333aa8f40554f35b2aed5d78ca
Program ID: Lighton6oQpVkeewmo2mcPTQQp7kYHr4fWpAgJyEmDX
Audit Result:
• **Audited Commit:** 1ee21c0976876def2e6f574116baa1e82c52d7f5

Item 4: Compressed Token Program V2
Link: https://github.com/Lightprotocol/light-protocol
Commit: 13c64a1f3c6205333aa8f40554f35b2aed5d78ca
Program ID: cTokenmWW8bLPjZEBAUgYy3zKxQZW6VKi7bqNFEVv3m
Audit Result:
• **Audited Commit:** 1ee21c0976876def2e6f574116baa1e82c52d7f5

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ISSUES SUMMARY

ID	TITLE	SEVERITY	STATUS
ACC-C1	Anchor IDL Buffer Feature Allows Creating Arbitrary Token Accounts	CRITICAL	FIXED
ACC-H1	Batched address trees never set `queue_index` (all inserts target queue 0)	HIGH	FIXED
ACC-H2	Batched Address Tree rollover fee can be redirected to an arbitrary account (fee-payer drain)	HIGH	FIXED
ACC-H3	Output Address Uniqueness Check can be Bypassed	HIGH	FIXED
ACC-M1	`migrate_state` clones the forester epoch account, dropping `work_counter` accounting updates	MEDIUM	FIXED
ACC-M2	Fee-based state trees can set a privileged `forester` and bypass epoch eligibility/min-weight gating	MEDIUM	FIXED
ACC-M3	store_data may be caching incorrect owner	MEDIUM	FIXED
ACC-M4	Close-and-Compress Can Close Accounts With Tokens and Cause Fund Loss	MEDIUM	FIXED
ACC-M5	Transfer2 Can Charge Extra Write Up Fees Because Lamports Move After All Compressions	MEDIUM	FIXED
ACC-M6	Missing Self-Transfer Check Breaks `ctoken_transfer`	MEDIUM	FIXED
ACC-M7	Missing Rent-Exemption Check Allows Creation of Invalid cToken Accounts	MEDIUM	FIXED
ACC-M8	Missing Mint Initialization and Owner Check Allows Creating Invalid cToken Accounts	MEDIUM	FIXED
ACC-M9	Close Authority Is Ignored When Closing Token Accounts	MEDIUM	FIXED
ACC-M10	Denial of Service via Account Balance Manipulation In `create_pda_account`	MEDIUM	FIXED
ACC-M11	Missing ATA Derivation Check Allows Creating Invalid Associated Token Accounts	MEDIUM	FIXED
ACC-L1	Metadata Extension Doesn't Account for Update after Removal in Action Batches	LOW	FIXED
ACC-L2	IDEMPOTENT ATA Creation Misses Owner and Mint Checks	LOW	FIXED
ACC-I1	Panic that should be an error	INFO	FIXED
ACC-I2	Multiple mutable references to the same account leads to undefined behavior	INFO	FIXED
ACC-I3	Ambiguous Use of `0` for `max_top_up` Creates Integration Risks	INFO	FIXED

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I4	Rent Sponsor Not Validated To Not Be The Token Account	INFO	FIXED

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

DETAILED ISSUES

ACC-C1 Anchor IDL Buffer Feature Allows Creating Arbitrary Token Accounts

CRITICAL FIXED

Description

The cToken program is a Pinocchio rewrite or patch to the existing Anchor cToken program. As we understand, the new program will be deployed at the address of the old Anchor program. This means that it will inherit some Anchor features, or on-chain state that could be created using Anchor features. In particular, Anchor implements a feature that allows users to upload an IDL to a predetermined PDA account. This feature has a side effect of allowing users to create a buffer account which is owned by the program. This buffer account has a discriminator and an arbitrary size of at least 44 bytes.

https://accretion.xyz/blog/hidden-idl-instructions

Now, this means that we can use this anchor feature to create a buffer account which belongs to the cToken program with a total length of 165. Into this buffer, we would crucially store the following data:

```
data[108] = 0;
data[64..72]: u64 = some high number
```

Then, we would use this prepared buffer account to initialize a new Light token account at this same address, pretending the buffer account is an empty new account.

For this we choose the non-compressible option, and the `initialize_ctoken_account` instruction will only check the account length, program ownership, and that data[108] is 0.

After this, it will set an arbitrary mint and ownership for us, and crucially not overwrite the amount (which we had set to some high number), assuming that it was 0.

It completely ignores the discriminator, as token accounts don't have discriminators at all.

The end result: A newly initialized cToken account, with an arbitrary mint, arbitrary owner and an arbitrary token amount as well.

Location

https://github.com/Lightprotocol/light-protocol/blob/13c64a1f3c6205333aa8f40554f35b2aed5d78ca/programs/compressed-token/program/src/shared/initialize_ctoken_account.rs#L46-L67

Relevant Code

```
/// initialize_ctoken_account.rs L46-L67
    // Manually initialize the token account at the correct offsets
    // SPL Token Account Layout (165 bytes total):
    // mint: 32 bytes (offset 0-31)
    // owner: 32 bytes (offset 32-63)
    // state: 1 byte (offset 108)
    // Account is already zeroed, only need to set these 3 fields

    let (base_token_bytes, extension_bytes) = token_account_data.split_at_mut(165);

    if base_token_bytes[108] != 0 {
        msg!("Token account already initialized");
        return Err(ErrorCode::AlreadyInitialized.into());
    }
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
    // Copy mint (32 bytes at offset 0)
    base_token_bytes[0..32].copy_from_slice(mint_pubkey);

    // Copy owner (32 bytes at offset 32)
    base_token_bytes[32..64].copy_from_slice(owner_pubkey);

    // Set state to Initialized (1 byte at offset 108)
    base_token_bytes[108] = 1;
```

Mitigation Suggestion

This issue becomes active when the program is ported to the new version. In this case there are 2 scenarios:

- new program doesn't support IdlBuffer creation anymore
- new program still supports IdlBuffer creation

If the new program doesn't support IdlBuffer creation anymore, we need to make sure that no malicious IDL Buffers (backdoors) have been created before the migration.

If the new program shall keep the feature, we need to implement proper validation of new token accounts. We could check that all bytes are 0, or overwrite all fields in the token account on initialization.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/0f5e5db49736c5eb7b568ad8595b111d1733fc3f.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-H1 Batched address trees never set `queue_index` (all inserts target queue 0)

HIGH FIXED

Description

For `AcpAccount::BatchedAddressTree`, `derive_new_addresses` only populates `InsertAddressInput.tree_index` and **never** sets `InsertAddressInput.queue_index`. Account Compression's batched address insertion path uses `queue_index` (and treats `tree_index` as unused), so every batched address insert is routed to **queue index 0**, regardless of the intended `address_merkle_tree_account_index`.

Impact

Unauthorized state transition / integrity break; when a tx includes new addresses spanning multiple batched address trees, the system program verifies non-inclusion against one set of roots but mutates a different tree/queue (whichever is at CPI index 0).

Location

https://github.com/Lightprotocol/light-protocol/blob/b5f7f10ba2f9982c3ec7eb1d1bb1345ddd379ad3/programs/account-compression/src/processor/insert_addresses.rs#L40-L48 https://github.com/Lightprotocol/light-protocol/blob/b5f7f10ba2f9982c3ec7eb1d1bb1345ddd379ad3/programs/system/src/processor/create_address_cpi_data.rs#L60-L94

Relevant Code

```
// programs/system/src/processor/create_address_cpi_data.rs:28-94
                .get(new_address_params.address_merkle_tree_account_index() as usize)
                .ok_or(SystemProgramError::NewAddressParamsIndexOutOfBounds)? {
            AcpAccount::AddressTree(tree) => {
                let address = derive_address_legacy::<Poseidon>(
                    &new_address_params.seed(),
                    tree.pubkey().as_ref(),
                    new_address_params.owner(),
                )
                .map_err(|_| SystemProgramError::DeriveAddressError)?;
                let rollover_fee = tree.metadata.rollover_metadata.rollover_fee;
                context.set_address_fee(
                    tree.metadata.rollover_metadata.network_fee,
                    new_address_params.address_merkle_tree_account_index(),
                )?;
                cpi_ix_data.addresses[i].tree_index = context.get_index_or_insert(
                    new_address_params.address_merkle_tree_account_index(),
                    remaining_accounts,
                    "V1 address tree",
                )?;
                cpi_ix_data.addresses[i].queue_index = context.get_index_or_insert(
                    new_address_params.address_queue_index(),
                    remaining_accounts,
                    "V1 address queue",
                )?;
                cpi_ix_data.insert_address_sequence_number(
                    &mut seq_index,
                    tree.pubkey().to_bytes(),
                    tree.queue_batches.next_index,
                );
                (address, rollover_fee)
            }
            AcpAccount::BatchedAddressTree(tree) => {
                let invoking_program_id_bytes = new_address_params
                    .owner()
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
                    .ok_or(SystemProgramError::DeriveAddressError)?
                    .to_bytes();
                let address = derive_address::<Poseidon>(
                    &new_address_params.seed(),
                    tree.pubkey().as_ref(),
                    invoking_program_id_bytes.as_ref(),
                );
                cpi_ix_data.addresses[i].tree_index = context.get_index_or_insert(
                    new_address_params.address_merkle_tree_account_index(),
                    remaining_accounts,
                    "V2 address tree",
                )?;
                // BUG: `queue_index` is NEVER set for batched address trees.
                // Account Compression uses `queue_index` for batched inserts; leaving it at 0
                // routes all inserts to CPI account index 0 (wrong tree/queue).
                let rollover_fee = tree.metadata.rollover_metadata.rollover_fee;
                context.set_address_fee(
                    tree.metadata.rollover_metadata.network_fee,
                    new_address_params.address_merkle_tree_account_index(),
                )?;
                cpi_ix_data.insert_address_sequence_number(
                    &mut seq_index,
                    tree.pubkey().to_bytes(),
                    tree.queue_batches.next_index,
                );
                (address, rollover_fee)
            }
            _ => return Err(SystemProgramError::InvalidAddressTreeAccount.into()),
        };

        context.set_rollover_fee(new_address_params.address_queue_index(), rollover_fee);

        if ADDRESS_ASSIGNMENT {


/// insert_addresses.rs L40-L48
    for &(tree_index, queue_index) in &visited {
        let queue_account = &mut accounts[queue_index as usize];

        match queue_account {
            AcpAccount::BatchedAddressTree(address_tree) => {
                inserted_addresses +=
                    batched_addresses(address_tree, addresses, queue_index, current_slot)?;
                anchor_lang::Result::Ok(())
            }
```

Mitigation Suggestion

Set the `queue_index`.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/782f906f9931148acb8e29d807c9460109ae63eb.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-H2 Batched Address Tree rollover fee can be redirected to an arbitrary account (fee-payer drain)

HIGH FIXED

Description

In the **batched address tree** branch of `derive_new_addresses`, the program **charges the tree's rollover fee to** `address_queue_index` from instruction data — **but never validates that this index actually points to the batched address tree account** (or any ACP-owned account). As a result, a caller can set `address_queue_index` to *any writable account* and the system program will later transfer the rollover fee from `fee_payer` to that arbitrary account.

Impact

Exploitable whenever a third-party fee sponsor/relayer is used (or any `fee_payer` not controlled by the attacker).

Location

programs/system/src/processor/create_address_cpi_data.rs:114-115

Relevant Code

```
// programs/system/src/processor/create_address_cpi_data.rs:114-115
114     // pays rollover fee once per tree
115     context.set_rollover_fee(new_address_params.address_queue_index(), rollover_fee);
        // ^ BUG: for BatchedAddressTree branch, address_queue_index is never validated
        //       and may point to an arbitrary writable account, redirecting fees.
```

Mitigation Suggestion

Validate the address queue index.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/782f906f9931148acb8e29d807c9460109ae63eb

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-H3 Output Address Uniqueness Check can be Bypassed

HIGH FIXED

Description

The system program tries to enforce “an address can only be used once in the outputs of a single instruction” by tracking a Vec<Option<Pubkey>> (context.addresses) and removing an address from that vector once it is consumed by an output.

However, the removal index is computed on a filtered iterator `(filter(|x| x.is_some()))` and then applied to the original vector `(context.addresses.remove(position))`. If there are any `None` entries before the target address, the position returned is not the correct index in the original vector. This causes the code to remove the wrong element (for example a None) and leaves the address still available, allowing the same address to be used multiple times across outputs.

Location

https://github.com/Lightprotocol/light-protocol/blob/13c64a1f3c6205333aa8f40554f35b2aed5d78ca/programs/system/src/processor/create_outputs_cpi_data.rs#L172-L185

Relevant Code

```
/// create_outputs_cpi_data.rs L172-L185
        // Check 3.
        if let Some(address) = account.address() {
            if let Some(position) = context
                .addresses
                .iter()
                .filter(|x| x.is_some())
                .position(|&x| x.unwrap() == address)
            {
                context.addresses.remove(position);
            } else {
                msg!(format!("context.addresses: {:?}", context.addresses).as_str());
                return Err(SystemProgramError::InvalidAddress.into());
            }
        }
```

Mitigation Suggestion

Calculate the position on an unfiltered vector.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/b2d4fc131d8d792e4f92193f0ba554d16607fca1.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M1 `migrate_state` clones the forester epoch account, dropping `work_counter` accounting updates

MEDIUM FIXED

Description

`migrate_state` passes a **cloned** `registered_forester_pda` into `check_forester`, so the `work_counter += …` update happens on the clone and is not persisted. This lets foresters perform work that is never reflected in epoch accounting (`work_counter` / `total_work`).

Impact

An eligible forester can execute `migrate_state` without the work being counted, skewing epoch totals and any reward distribution/metrics based on `work_counter`.

Location

`programs/registry/src/lib.rs:675-688` (`migrate_state`)

Relevant Code

```
// programs/registry/src/lib.rs:675-688
675 pub fn migrate_state<'info>(
676     ctx: Context<'_, '_, '_, 'info, MigrateState<'info>>,
677     bump: u8,
678     inputs: MigrateLeafParams,
679 ) -> Result<()> {
680     check_forester(
681         &ctx.accounts.merkle_tree.load()?.metadata,
682         ctx.accounts.authority.key(),
683         ctx.accounts.merkle_tree.key(),
684         &mut Some(ctx.accounts.registered_forester_pda.clone()), // ISSUE: clone => work_counter incremen
ts are discarded
685         DEFAULT_WORK_V1,
686     )?;
687     process_migrate_state(&ctx, bump, inputs)
688 }
```

Mitigation Suggestion

Don't pass a clone, or persist the changes made on the clone.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/be9fa48c129c2b5feae0c9bb6c9fd6a47a3fd825.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M2 Fee-based state trees can set a privileged `forester` and bypass epoch eligibility/min-weight gating

MEDIUM FIXED

Description

`initialize_state_merkle_tree` fails to forbid a non-`None` `forester` when `network_fee` is set (fee-based tree), and `check_forester` lets `metadata.access_metadata.forester` bypass registered-forester eligibility checks **without requiring** `network_fee == 0`. This contradicts the documented intent that privileged `forester` is for **custom trees without a network fee**.

Impact

Any user can create a fee-based state tree charging the standard network fee but designate themselves as `forester`, then perform forester-only maintenance without registering, meeting `min_weight`, or passing slot-based eligibility (and can potentially monopolize fees/rewards for that tree).

Location

• [`programs/registry/src/lib.rs:325-370`](https://github.com/Lightprotocol/light-protocol/blob/b5f7f10ba2f9982c3ec7eb1d1bb1345ddd379ad3/programs/registry/src/lib.rs#L325-L387)

) (`initialize_state_merkle_tree`)

• [`programs/registry/src/lib.rs:806-833`](https://github.com/Lightprotocol/light-protocol/blob/b5f7f10ba2f9982c3ec7eb1d1bb1345ddd379ad3/programs/registry/src/lib.rs#L806-L833)

) (`check_forester`)

• [`program-libs/merkle-tree-metadata/src/access.rs:23-35`](https://github.com/Lightprotocol/light-protocol/blob/b5f7f10ba2f9982c3ec7eb1d1bb1345ddd379ad3/program-libs/merkle-tree-metadata/src/access.rs#L23-L35)

) (`AccessMetadata::forester` docs)

• (intent signal) [`programs/registry/src/lib.rs:506-513`](https://github.com/Lightprotocol/light-protocol/blob/b5f7f10ba2f9982c3ec7eb1d1bb1345ddd379ad3/programs/registry/src/lib.rs#L506-L516)

) (batched state init explicitly forbids `forester` with `network_fee`)

Relevant Code

```
// programs/registry/src/lib.rs:325-370
pub fn initialize_state_merkle_tree(
    ctx: Context<InitializeMerkleTreeAndQueue>,
    bump: u8,
    program_owner: Option<Pubkey>,
    forester: Option<Pubkey>,
    merkle_tree_config: StateMerkleTreeConfig,
    queue_config: NullifierQueueConfig,
) -> Result<()> {
    // Program owned trees are disabled
    if program_owner.is_some() {
        msg!("Program owner must not be defined.");
        return err!(RegistryError::ProgramOwnerDefined);
    }
    // The network fee must be either zero or the same as the protocol config.
    // Only trees with a network fee will be serviced by light foresters.
    if let Some(network_fee) = merkle_tree_config.network_fee {
        if network_fee != ctx.accounts.protocol_config_pda.config.network_fee {
            return err!(RegistryError::InvalidNetworkFee);
        }
        // ISSUE: missing `require!(forester.is_none())` for fee-based trees
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
345         //        (batched variant enforces this, see below)
346     } else if forester.is_none() {
347         /* ... */
348     }
349     /* ... */
362     process_initialize_state_merkle_tree(
363         &ctx,
364         bump,
365         0,
366         program_owner,
367         forester, // ISSUE: allows setting privileged forester even when network_fee is set
368         merkle_tree_config,
369         queue_config,
370     )?;
371     /* ... */

// programs/registry/src/lib.rs:806-833
806 pub fn check_forester(
807     metadata: &MerkleTreeMetadata,
808     authority: Pubkey,
809     queue: Pubkey,
810     registered_forester_pda: &mut Option<Account<'_, ForesterEpochPda>>,
811     num_work_items: u64,
812 ) -> Result<()> {
813     if let Some(forester_pda) = registered_forester_pda.as_mut() {
814         ForesterEpochPda::check_forester_in_program(/* ... */)?;
824         if metadata.rollover_metadata.network_fee == 0 {
825             return err!(RegistryError::InvalidNetworkFee);
826         }
827         Ok(())
828     } else if metadata.access_metadata.forester == authority {
829         Ok(()) // ISSUE: bypasses eligibility even if `metadata.rollover_metadata.network_fee != 0`
830     } else {
831         err!(RegistryError::InvalidSigner)
832     }
833 }

// program-libs/merkle-tree-metadata/src/access.rs:23-35
23 pub struct AccessMetadata {
24     /// Owner of the Merkle tree.
25     pub owner: Pubkey,
26     /// Program owner of the Merkle tree. This will be used for program owned Merkle trees.
27     pub program_owner: Pubkey,
28     /// Optional privileged forester pubkey, can be set for custom Merkle trees
29     /// without a network fee. Merkle trees without network fees are not
30     /// forested by light foresters. The variable is not used in the account
31     /// compression program but the registry program. The registry program
32     /// implements access control to prevent contention during forester. The
33     /// forester pubkey specified in this struct can bypass contention checks.
34     pub forester: Pubkey,
35 }

// programs/registry/src/lib.rs:506-513 (batched state init intent)
506 if let Some(network_fee) = params.network_fee {
507     if network_fee != ctx.accounts.protocol_config_pda.config.network_fee { /* ... */ }
510     if params.forester.is_some() {
511         msg!("Forester pubkey must not be defined for trees serviced by light foresters.");
512         return err!(RegistryError::ForesterDefined); // intent: fee-based trees shouldn't have privileged
 forester
513     }
```

Mitigation Suggestion

Apply the same logic as for batched state merkle trees.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/845c4d39c2a77f732e8ea47811462101985bc2d0.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M3 store_data may be caching incorrect owner

MEDIUM FIXED

Description

We found that the store_data function tries to get the owner for newly created addresses using `instruction_data.owner()`. However, in the relevant implementation, this function will return the owner of the first input account, or a zero pubkey if no input accounts exist. This means that if a program would create new addresses without consuming any input accounts, it may assign the new addresses to an incorrect zero-owner.

Location

https://github.com/Lightprotocol/light-protocol/blob/13c64a1f3c6205333aa8f40554f35b2aed5d78ca/programs/system/src/cpi_context/state.rs#L136-L136

Relevant Code

```
/// state.rs L136-L136
        let owner_bytes = instruction_data.owner().to_bytes();
```

Mitigation Suggestion

The owner of the new addresses should be the program itself, even when no inputs are passed. We need to get that address from somewhere else.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/pull/2277.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M4 Close-and-Compress Can Close Accounts With Tokens and Cause Fund Loss

MEDIUM FIXED

Description

We found that the close-and-compress flow does **not** validate that a token account has **zero tokens** before closing it. Because of this, in `transfer2`, an account can be compressed and then closed even if it ends up holding tokens later in the same instruction.

This can cause **loss of funds**.

Scenario

- We request to `close_and_compress` token account **A** (A initially has **100 tokens**).
- In the same `transfer2`, we also transfer **200 tokens** from **B** → **A**.
- With the current logic, the instruction may:

1. Execute transfers, 2. Then run close-and-compress, 3. And close/zero account **A** even if it now holds tokens (e.g., **200 tokens** after a decompress step).

So the program can close an account with a **positive token balance**.

Location

https://github.com/Lightprotocol/light-protocol/blob/5f9c172928ce59883202b447643b3de06f3f68f2/programs/compressed-token/program/src/transfer2/compression/ctoken/compress_and_close.rs#L230

Relevant Code

Mitigation Suggestion

Before allowing close-and-compress, enforce **zero token balance**:

- Check `token_account.amount == 0` right before closing.
- If the amount is not zero, fail the instruction with a clear error (e.g., `CannotCloseWithBalance`).

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/fd8026bea.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M5 Transfer2 Can Charge Extra Write Up Fees Because Lamports Move After All Compressions

MEDIUM FIXED

Description

We found that users may pay more top-up (write-up) fee than needed in the `transfer2` instruction. This happens because lamport transfers are done **after** all compressions, instead of happening per transfer.

Example scenario:

- There are three token accounts: **A**, **B**, **C**
- We transfer **1 token from A** → **C** and **2 tokens from B** → **C**
- Token account **C** is compressible and has a **rent deficit**

Because lamport top-ups happen only after all compressions, the logic may treat each incoming transfer as needing to cover the rent deficit again. This can lead to paying the deficit multiple times. In this example, the user ends up paying **2 × rent_deficit**.

If the program instead performs `multi_transfer_lamports` **after each transfer**, the rent deficit would be covered once, and the user would pay only **1 × rent_deficit**.

Location

https://github.com/Lightprotocol/light-protocol/blob/5f9c172928ce59883202b447643b3de06f3f68f2/programs/compressed-token/program/src/transfer2/compression/mod.rs#L68-L132

Relevant Code

```
match source_or_recipient.owner() {
                ID => ctoken::process_ctoken_compressions(
                    inputs,
                    compression,
                    source_or_recipient,
                    packed_accounts,
                    &mut transfer_map[account_index],
                    &mut lamports_budget,
                )?,
                SPL_TOKEN_ID => {
                    spl::process_spl_compressions(
                        compression,
                        &SPL_TOKEN_ID.to_pubkey_bytes(),
                        source_or_recipient,
                        packed_accounts,
                        cpi_authority,
                    )?;
                }
                SPL_TOKEN_2022_ID => {
                    spl::process_spl_compressions(
                        compression,
                        &SPL_TOKEN_2022_ID.to_pubkey_bytes(),
                        source_or_recipient,
                        packed_accounts,
                        cpi_authority,
                    )?;
                }
                _ => {
                    msg!(
                        "source_or_recipient {:?}",
                        solana_pubkey::Pubkey::new_from_array(*source_or_recipient.key())
                    );
                    msg!(
```

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

```
                    "Invalid token program ID {:?}",
                    solana_pubkey::Pubkey::from(*source_or_recipient.owner())
                );
                return Err(ProgramError::InvalidInstructionData);
            }
        };
    }

    let transfers: ArrayVec<Transfer, MAX_PACKED_ACCOUNTS> = transfer_map
        .iter()
        .enumerate()
        .filter_map(|(index, &amount)| {
            if amount != 0 {
                Some((index as u8, amount))
            } else {
                None
            }
        })
        .map(|(index, amount)| {
            Ok(Transfer {
                account: packed_accounts.get_u8(index, "transfer account")?,
                amount,
            })
        })
        .collect::<Result<ArrayVec<Transfer, MAX_PACKED_ACCOUNTS>, ProgramError>>()?;

    if !transfers.is_empty() {
        // Check budget wasn't exhausted (0 means exceeded max_top_up)
        if max_top_up != 0 && lamports_budget == 0 {
            return Err(CTokenError::MaxTopUpExceeded.into());
        }
        multi_transfer_lamports(fee_payer, &transfers).map_err(convert_program_error)?
    }
```

Mitigation Suggestion

Move lamport top-up logic closer to each transfer, run `multi_transfer_lamports` **after each individual transfer** (or update rent deficit state immediately after the first top-up).

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/pull/2090/.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M6 Missing Self-Transfer Check Breaks `ctoken_transfer`

MEDIUM FIXED

Description

We found that `ctoken_transfer` does **not** handle **self transfers** (when `source == destination`). This causes two related issues:

1. **Instruction Failure (UnbalancedInstruction)** During a self transfer, the code still tries to move lamports using `transfer_lamports`. Since self transfers are not handled, the Solana runtime detects that the total lamports across the instruction's accounts do not net to zero and throws an `UnbalancedInstruction` error. As a result, the transaction always fails.

2. **Incorrect Extra Top-Up Calculation** When `source == destination`, the logic counts the required `top_up_lamports` **twice**, even though no real transfer is needed. This leads to overestimation of required lamports and incorrect behavior.

Together, these issues make self transfers fail and cause incorrect lamport accounting.

Location

https://github.com/Lightprotocol/light-protocol/blob/9672e086ba4e449dff29780b15cdf049e24f831c/programs/compressed-token/program/src/shared/transfer_lamports.rs#L34-L35

https://github.com/Lightprotocol/light-protocol/blob/9672e086ba4e449dff29780b15cdf049e24f831c/programs/compressed-token/program/src/shared/transfer_lamports.rs#L68-L73

Relevant Code

```
*from.try_borrow_mut_lamports()? = from_lamports;
    *to.try_borrow_mut_lamports()? = to_lamports;


    // Calculate total amount needed
    let total_amount: u64 = transfers
        .iter()
        .map(|t| t.amount)
        .try_fold(0u64, |acc, amt| acc.checked_add(amt))
        .ok_or(ProgramError::ArithmeticOverflow)?;
```

Mitigation Suggestion

Consider self-transfer in ctoken transfer.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/8835b7255133840da945cb4987085fad4d1f2e0d and https://github.com/Lightprotocol/light-protocol/commit/21d7863b07b697d9edd625f3c58df8447ce488ea.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M7 Missing Rent-Exemption Check Allows Creation of Invalid cToken Accounts

MEDIUM FIXED

Description

We found that `initialize_ctoken_account` does **not** check whether the created token account is **rent-exempt**. Because of this, a user can create a token account with **0 lamports**. Such an account will later be **garbage-collected by the runtime**.

The behaviour is incompatible with standard SPL Token requirements, which *always* expect token accounts to be rent-exempt, and it creates attack vectors.

Location

https://github.com/Lightprotocol/light-protocol/blob/06f3122d1756ab7c08aa23b62b958a04b631fdb1/programs/compressed-token/program/src/shared/initialize_ctoken_account.rs#L32-L33

Relevant Code

```
// Access the token account data as mutable bytes
    let mut token_account_data = AccountInfoTrait::try_borrow_mut_data(token_account_info)?;
```

Mitigation Suggestion

Require that the cToken account meet rent-exemption requirements before completing initialization.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/21d7863b07b697d9edd625f3c58df8447ce488ea.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M8 Missing Mint Initialization and Owner Check Allows Creating Invalid cToken Accounts

MEDIUM FIXED

Description

We found that `initialize_ctoken_account` does **not** verify whether the provided mint account is actually initialized and owned by the program. Because of this missing check, a user can create a cToken account that points to a mint account that does not exist or is not a valid SPL mint.

- The created token account is incompatible with SPL Token program expectations.
- Downstream logic that assumes a valid mint can break.

Location

https://github.com/Lightprotocol/light-protocol/blob/06f3122d1756ab7c08aa23b62b958a04b631fdb1/programs/compressed-token/program/src/shared/initialize_ctoken_account.rs#L60-L61

Relevant Code

```
// Copy mint (32 bytes at offset 0)
    base_token_bytes[0..32].copy_from_slice(mint_pubkey);
```

Mitigation Suggestion

Before initializing a cToken account, add a validation step to ensure the mint account:

- Exists,
- Is owned by the SPL Token program,
- Is a properly initialized mint.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/pull/2251.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M9 Close Authority Is Ignored When Closing Token Accounts

MEDIUM FIXED

Description

We found that `close_token_account::validate_token_account` does not check the `close_authority` field of the token account.

If a token account has a `close_authority` set, only that address should be allowed to close it. However, because this check is missing, the **main authority (owner)** can still close the account, and the configured `close_authority` cannot use its role as intended. This breaks the expected SPL Token behavior and can lead to incorrect permission handling.

Location

https://github.com/Lightprotocol/light-protocol/blob/5839134eef318a07e05d3fd12c56ca112232d77f/programs/compressed-token/program/src/close_token_account/processor.rs#L77-L78

Relevant Code

```
// Verify the authority matches the account owner or rent authority (if compressible)
    let owner_matches = ctoken.owner.to_bytes() == *accounts.authority.key();
```

Mitigation Suggestion

Update `validate_token_account` to:

- Check if `close_authority` is set on the token account.
- If it is set, require the signer to match the `close_authority`.
- Only fall back to checking the main authority when `close_authority` is `None`.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/7119a91dd.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M10 Denial of Service via Account Balance Manipulation In `create_pda_account`

MEDIUM FIXED

Description

We found that an attacker could send some lamports to `new_account` and cause a denial of service (DoS) in `create_pda_account`. This happens because CreateAccount reverts if the target account already has a positive balance. You can see this behavior in the system processor implementation here.

Location

https://github.com/Lightprotocol/light-protocol/blob/5839134eef318a07e05d3fd12c56ca112232d77f/programs/compressed-token/program/src/shared/create_pda_account.rs#L52-L70

Relevant Code

```
let create_account = CreateAccount {
        from: fee_payer,
        to: new_account,
        lamports,
        space: account_size as u64,
        owner: &LIGHT_CPI_SIGNER.program_id,
    };


    let mut signers = arrayvec::ArrayVec::<Signer, N>::new();
    for seeds in seeds_inputs.iter() {
        if !seeds.is_empty() {
            signers.push(Signer::from(*seeds));
        }
    }


    create_account
        .invoke_signed(signers.as_slice())
        .map_err(convert_program_error)
}
```

Mitigation Suggestion

Instead of using CreateAccount, calculate the required lamports, transfer them to the target account, and then perform allocate and assign operations manually.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/fbbd1fe37b76cc4e229cac453bcd2a9e67679abb.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-M11 Missing ATA Derivation Check Allows Creating Invalid Associated Token Accounts

MEDIUM FIXED

Description

We found that `process_create_associated_token_account_inner` does **not** validate whether the provided `associated_token_account` matches the **PDA-derived ATA address** in not IDEMPOTENT branch. Because of this missing check, a user can:

- Provide **any keypair** as the ATA account,
- Sign the transaction to satisfy the create-account requirement,
- And end up creating an **ATA on an arbitrary keypair**, not the correct PDA.

Since signature validation happens at the transaction level, the on-chain program **does not reject** the creation attempt, even though the supplied address is not the true PDA ATA.

Location

https://github.com/Lightprotocol/light-protocol/blob/5839134eef318a07e05d3fd12c56ca112232d77f/programs/compressed-token/program/src/create_associated_token_account.rs#L74-L89

Relevant Code

```
let fee_payer = iter.next_signer_mut("fee_payer")?;
    let associated_token_account = iter.next_mut("associated_token_account")?;
    let _system_program = iter.next_non_mut("system_program")?;

    // If idempotent mode, check if account already exists
    if IDEMPOTENT {
        // Verify the PDA derivation is correct
        validate_ata_derivation(associated_token_account, owner_bytes, mint_bytes, bump)?;
        // If account is already owned by our program, it exists - return success
        if associated_token_account.is_owned_by(&crate::LIGHT_CPI_SIGNER.program_id) {
            return Ok(());
        }
    }
```

Mitigation Suggestion

Always perform `validate_ata_derivation` for BOTH modes:

- **IDEMPOTENT**
- **Non-IDEMPOTENT**

This ensures the provided `associated_token_account` always matches the correct PDA ATA and prevents users from creating ATAs at arbitrary addresses.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/291933b232deaeac955ab750783a2caf4e34ece9.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L1 Metadata Extension Doesn't Account for Update after Removal in Action Batches

LOW FIXED

Description

We found that by creating an actions array with the actions

- UpdateMetadataField
- RemoveMetadataKey
- UpdateMetadataField

(Adding a key, removing it, and then adding it again)

We could confuse the `should_add_key` function. In theory, the net result is that the key should be added. However in practice, the function will return false, because it doesn't take the order of operations into account. In general, it seems that if a key is added after it is removed, the function gets confused.

Location

https://github.com/Lightprotocol/light-protocol/blob/13c64a1f3c6205333aa8f40554f35b2aed5d78ca/programs/compressed-token/program/src/extensions/mod.rs#L128-L148

Relevant Code

```
/// mod.rs L128-L148
    let should_add_key = |key: &[u8]| -> bool {
        // Key exists if it's in original metadata OR added via UpdateMetadataField
        let exists_in_original =
            metadata.is_some_and(|items| items.iter().any(|item| item.key == key));
        let added_via_update = actions.iter().any(|action| {
            matches!(action, ZAction::UpdateMetadataField(update)
                if update.extension_index as usize == extension_index
                    && update.field_type == 3
                    && update.key == key)
        });

        // Key should be included if it exists and is not removed
        let should_exist = exists_in_original || added_via_update;
        let is_removed = actions.iter().any(|action| {
            matches!(action, ZAction::RemoveMetadataKey(remove)
                if remove.extension_index as usize == extension_index
                    && remove.key == key)
        });

        should_exist && !is_removed
    };
```

Mitigation Suggestion

This could be intentional (no additions after removals), but then should be documented, or checked for. Otherwise, the algorithm should be adjusted to account for additions after removals.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/195a97f31fbe1b34f7c784687d066227e7236486.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-L2 IDEMPOTENT ATA Creation Misses Owner and Mint Checks

LOW FIXED

Description

We found that in `process_create_associated_token_account_inner` when running in **IDEMPOTENT** mode, the code only checks the PDA derivation and whether the account is owned by `LIGHT_CPI_SIGNER`, but **does not** verify the **mint** or **owner** fields like the on-chain ATA program does.

Because of this, an **empty** `associated_token_account` (with zero data and zero balance) can pass the check even if its internal **mint** or **owner** fields are wrong. If its balance stays 0, it can later be reset after the transaction, but it already bypassed the validation once. This can lead to incorrect or unsafe assumptions about who really owns the ATA and which mint it belongs to.

Location

https://github.com/Lightprotocol/light-protocol/blob/5839134eef318a07e05d3fd12c56ca112232d77f/programs/compressed-token/program/src/create_associated_token_account.rs#L81-L89

Relevant Code

```
// If idempotent mode, check if account already exists
    if IDEMPOTENT {
        // Verify the PDA derivation is correct
        validate_ata_derivation(associated_token_account, owner_bytes, mint_bytes, bump)?;
        // If account is already owned by our program, it exists - return success
        if associated_token_account.is_owned_by(&crate::LIGHT_CPI_SIGNER.program_id) {
            return Ok(());
        }
    }
```

Mitigation Suggestion

In IDEMPOTENT mode, add full checks like the standard ATA program:

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/21d7863b07b697d9edd625f3c58df8447ce488ea.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I1 Panic that should be an error

INFO FIXED

Description

There a checked_sub that will panic if the rent requirement of a new merkle tree exceeds the lamports of the old one on a rollover. It should instead go through to the error in the next line.

Location

https://github.com/Lightprotocol/light-protocol/blob/13c64a1f3c6205333aa8f40554f35b2aed5d78ca/program-libs/batched-merkle-tree/src/rollover_address_tree.rs#L20-L24

Relevant Code

```
/// rollover_address_tree.rs L20-L24
    let new_mt_rent = check_account_balance_is_rent_exempt(new_account, old_account.data_len())?;
    #[cfg(target_os = "solana")]
    if old_account.lamports().checked_sub(new_mt_rent).unwrap() == 0 {
        return Err(MerkleTreeMetadataError::NotReadyForRollover.into());
    }
```

Mitigation Suggestion

Do something like

```
if old_account.lamports() < new_mt_rent {
    Error
}
```

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/pull/2278.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I2 Multiple mutable references to the same account leads to undefined behavior

INFO FIXED

Description

We found that when the compressed token program falls back to the anchor implementation, it converts the AccountInfos from the Pinocchio type to the Anchor type, and while doing so wraps each AccountInfo's data in a `Rc<RefCell>` practically allowing Anchor to create multiple mutable references of the same data at the same time. This can happen if the same account is passed multiple times, which may be done by passing the same merkle tree account multiple times. In practice, this can lead to undefined behavior and the Rust compiler applying incorrect optimizations, which can lead to state corruption.

Location

https://github.com/Lightprotocol/light-protocol/blob/13c64a1f3c6205333aa8f40554f35b2aed5d78ca/programs/compressed-token/program/src/convert_account_infos.rs#L47-L47

Relevant Code

```
/// convert_account_infos.rs L47-L47
        let data = Rc::new(RefCell::new(pinocchio_account.borrow_mut_data_unchecked()));
```

Mitigation Suggestion

Duplicate accounts should either be rejected, or all duplicate occurrences should share the same borrow state so double-mutable-borrows are detected and prevented.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/94ae5ca630343817629f94a950725da6d4adff7f.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I3 Ambiguous Use of `0` for `max_top_up` Creates Integration Risks

INFO FIXED

Description

We found that `ctoken_transfer` uses the value `0` to mean **"no limit"** for `max_top_up`. This creates ambiguity and integration risk because `0` is doing double duty:

- It means **"not provided (legacy behavior)"**, and
- It also means **"explicitly no limit"**.

This is especially dangerous for integrators and CPI callers, since instruction data may be **zero-initialized by default**. As a result, other programs may unintentionally allow unlimited top-ups without realizing it.

This makes it easy to accidentally allow unlimited top-ups and hides intent in the instruction format.

Location

https://github.com/Lightprotocol/light-protocol/blob/9672e086ba4e449dff29780b15cdf049e24f831c/programs/compressed-token/program/src/ctoken_transfer.rs#L40-L49

Relevant Code

```
// 0 means no limit
    let max_top_up = match instruction_data.len() {
        8 => 0u16, // Legacy: no max_top_up
        10 => u16::from_le_bytes(
            instruction_data[8..10]
                .try_into()
                .map_err(|_| ProgramError::InvalidInstructionData)?,
        ),
        _ => return Err(ProgramError::InvalidInstructionData),
    };
```

Mitigation Suggestion

Redefine the meaning of `max_top_up` values:

- Use `u16::MAX` as the explicit sentinel for **"no limit"**.
- Reserve `0` to mean **"no top-ups allowed"** (fail if any top-up is required).

This removes ambiguity, makes intent explicit, and prevents accidental unlimited top-ups by integrators or CPI callers.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/46514947d10d0bed1dccb2b13b9037275e3fe57b.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

ACC-I4 Rent Sponsor Not Validated To Not Be The Token Account

INFO FIXED

Description

In the close token account, Although there is a check preventing the **destination** from being the token account, there is **no such check for the rent sponsor**, which leaves a gap in validation, and we found that during **token account and ATA creation**, the program does not check whether the **rent sponsor** is the same as the **token account being created**.

Example scenario:

1. Alice creates a compressible token account. 2. She sets the **rent sponsor = token account itself**. 3. The creation succeeds. 4. Later, during **close and compress**, the lamports remain. 5. The token account ends up holding a **rent-exempt balance**.

While this may not currently cause a critical exploit because we zeroed out data and assigned it to 0, it creates inconsistent behaviour and could become an attack vector in future updates.

Location

https://github.com/Lightprotocol/light-protocol/blob/13c64a1f3c6205333aa8f40554f35b2aed5d78ca/programs/compressed-token/program/src/create_token_account.rs#L187-L188

Relevant Code

Mitigation Suggestion

In creating a token account, check if the rent sponsor is the same as the token account or not.

Remediation

Fixed in https://github.com/Lightprotocol/light-protocol/commit/d3e93c0206ae00218450c17b4304fd05e05588da.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.

APPENDIX

Vulnerability Classification

We rate our issues according to the following scale. Informational issues are reported informally to the developers and are not included in this report.

Severity	Description
Critical	Vulnerabilities that can be easily exploited and result in loss of user funds, or directly violate the protocol's integrity. Immediate action is required.
High	Vulnerabilities that can lead to loss of user funds under non-trivial preconditions, loss of fees, or permanent denial of service that requires a program upgrade. These issues require attention and should be resolved in the short term.
Medium	Vulnerabilities that may be more difficult to exploit but could still lead to some compromise of the system's functionality. For example, partial denial of service attacks, or such attacks that do not require a program upgrade to resolve, but may require manual intervention. These issues should be addressed as part of the normal development cycle.
Low	Vulnerabilities that have a minimal impact on the system's operations and can be fixed over time. These issues may include inconsistencies in state, or require such high capital investments that they are not exploitable profitably.
Informational	Findings that do not pose an immediate risk but could affect the system's efficiency, maintainability, or best practices.

Audit Methodology

Accretion is a boutique security auditor specializing in Solana's ecosystem. We employ a customized approach for each client, strategically allocating our resources to maximize code review effectiveness. Our auditors dedicate substantial time to developing a comprehensive understanding of each program under review, examining design decisions, expected and edge-case behaviors, invariants, optimizations, and data structures, while meticulously verifying mathematical correctness—all within the context of the developers' intentions.

Our audit scope extends beyond on-chain components to include associated infrastructure, such as user interfaces and supporting systems. Every audit encompasses both a holistic protocol design review and detailed line-by-line code analysis.

During our assessment, we focus on identifying:
• Solana-specific vulnerabilities
• Access control issues
• Arithmetic errors and precision loss
• Race conditions and MEV opportunities
• Logic errors and edge cases
• Performance optimization opportunities
• Invariant violations
• Account confusion vulnerabilities
• Authority check omissions
• Token22 implementation risks and SPL-related pitfalls
• Deviations from best practices

Our approach transcends conventional vulnerability classifications. We continuously conduct ecosystem-wide security research to identify and mitigate emerging threat vectors, ensuring our audits remain at the forefront of Solana security practices.

© 2026 Accretion Labs Pte. Ltd. All Rights Reserved.